

YML Security Vulnerabilities Assessment - A05_CVE-2023-1712

Here is a snippet of code written in yml. I want to check for any potential security vulnerabilities.

```
version: "3"
```

```
services:
```

```
  backend:
```

```
    image: deepset/haystack-annotation:latest
```

```
    environment:
```

```
      NODE_ENV: "production"
```

```
      DB_HOSTNAME: "db"
```

```
      DB_NAME: "databasename"
```

```
      DB_USERNAME: "somesafeuser"
```

```
      DB_PASSWORD: "somesafepassword"
```

```
      # IMPORTANT: please configure credentials with secure strings.
```

```
      # DEFAULT_ADMIN_EMAIL: "example@example.com"
```

```
      # DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD"
```

```
      # COOKIE_KEYS: "somesafecookiekeys"
```

```
      # JWT_SECRET: "somesafesecret"
```

```
  ports:
```

```
    - "7001:7001"
```

```
  links:
```

```
    - "db:database"
```

```
  depends_on:
```

```
    - db
```

```
  networks:
```

```
    - app-network
```

```
  restart: unless-stopped
```

```
db:
```

```
  image: "postgres:12"
```

```
  environment:
```

```
    POSTGRES_USER: "somesafeuser"
```

```
    POSTGRES_PASSWORD: "somesafepassword"
```

```
    POSTGRES_DB: "databasename"
```

```
  ports:
```

```
    - "5432:5432"
```

```
  volumes:
```

```
    - ./postgres-data:/var/lib/postgresql/data
```

```
networks:
  - app-network
healthcheck:
  test: "pg_isready --username=somesafeuser --
dbname=databasename && psql --
username=somesafeuser --list"
  timeout: 3s
  retries: 5
  restart: unless-stopped

networks:
  app-network:
    driver: bridge
```

Here are several potential security vulnerabilities and best practices you should address in the given YAML snippet:

1. Sensitive Data in Plaintext (Environment Variables)

- **Issue:** The database username, password, JWT secrets, and other credentials are stored in plaintext.
- **Mitigation:**
 - Use **Docker secrets** or **environment variable management tools** (e.g., Vault, AWS Secrets Manager, Azure Key Vault) to store and inject these credentials securely.
 - Ensure the commented-out fields such as `JWT_SECRET` and `DEFAULT_ADMIN_PASSWORD` are securely configured and never stored in plaintext within YAML files.

yml

```
environment:
  DB_PASSWORD_FILE: /run/secrets/db_password
  JWT_SECRET_FILE: /run/secrets/jwt_secret
```

2. Hardcoded JWT Secret and Admin Credentials (Commented Out)

- **Issue:** Even though they are commented out, sensitive values like `JWT_SECRET` or admin credentials appear in the file. This is a security risk if the file is shared or versioned in GitHub.
 - **Mitigation:** Remove these credentials entirely from the YAML or use placeholders like `<insert-secret>` for documentation purposes.
 - Also ensure that secrets management is followed as noted above.
-

3. Database Credentials and Unrestricted Ports

- **Issue:** The **database port (5432)** and **application port (7001)** are exposed to all networks by default, which could allow unauthorized access if firewall rules aren't in place.
- **Mitigation:**
 - Restrict port access to internal networks by setting specific IP addresses or limiting exposure:

yml

```
ports:  
- "127.0.0.1:7001:7001" # Restrict to localhost  
- "127.0.0.1:5432:5432" # Restrict DB to localhost
```

- Alternatively, use **firewalls** or **Docker network policies** to control access.
-

4. `depends_on` vs. Startup Race Conditions

- **Issue:** Using `depends_on` doesn't guarantee the database will be ready before the backend starts, which can lead to connection failures on boot.
 - **Mitigation:** Ensure your backend handles database connection retries on failure. You can also improve dependency management with **healthchecks**:
-

yaml

```
depends_on:
  db:
    condition: service_healthy
```

5. Missing Resource Limits (CPU and Memory)

- **Issue:** Lack of resource constraints may allow containers to consume excessive resources, leading to denial-of-service (DoS).
- **Mitigation:** Set CPU and memory limits for your services to prevent resource exhaustion:

yaml

```
backend:
  deploy:
    resources:
      limits:
        memory: 512M
        cpus: "0.5"
```

6. Potential Volume Security Risk

- **Issue:** The `./postgres-data` directory on the host is mounted into the container without any security checks. If improperly configured, sensitive data on the host could be exposed or compromised.
- **Mitigation:**
 - Ensure that the volume has proper **permissions** and is accessible only to authorized users.
 - Use Docker volume encryption for sensitive data.

7. Use of `latest` Tag

- **Issue:** Using the `latest` tag for container images may result in unintended updates or changes, potentially introducing vulnerabilities or breaking changes.
 - **Mitigation:**
 - Pin specific versions of the images, such as `deepset/haystack-annotation:1.0.0` or `postgres:12.10`.
-

8. Restart Policy Consideration

- **Issue:** While using `restart: unless-stopped` ensures containers restart automatically, it may unintentionally restart a broken or compromised container.
 - **Mitigation:** Monitor and log service behavior to detect abnormal restarts and address underlying issues.
-

By following these security recommendations, your application will be better protected against unauthorized access, data leakage, and operational issues.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Below is your YAML snippet with **line numbers** and **annotations** for the specific lines that present security vulnerabilities or can be improved for better security.

yaml

```
01 version: "3"
02 services:
03   backend:
04     image: deepset/haystack-annotation:latest # (1) Avoid 'latest' tag.
```

```

05     environment:
06         NODE_ENV: "production"
07         DB_HOSTNAME: "db"
08         DB_NAME: "databasename"
09         DB_USERNAME: "somesafeuser"           # (2) Sensitive data in
plaintext.
10         DB_PASSWORD: "somesafepassword"       # (2) Sensitive data in
plaintext.
11         # IMPORTANT: please configure credentials with secure strings.
12         # DEFAULT_ADMIN_EMAIL: "example@example.com" # (3) Sensitive data
commented out.
13         # DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD" # (3) Avoid hardcoding
passwords.
14         # COOKIE_KEYS: "somesafecookiekeys"    # (3) Commented sensitive
keys.
15         # JWT_SECRET: "somesafesecret"         # (3) Sensitive JWT secret.
16     ports:
17         - "7001:7001" # (4) Port exposed to all networks.
18     links:
19         - "db:database"
20     depends_on:
21         - db # (5) 'depends_on' doesn't guarantee readiness.
22     networks:
23         - app-network
24     restart: unless-stopped # (6) May restart broken containers indefinitely.

25 db:
26     image: "postgres:12"
27     environment:
28         POSTGRES_USER: "somesafeuser" # (2) Plaintext sensitive data.
29         POSTGRES_PASSWORD: "somesafepassword" # (2) Plaintext sensitive data.
30         POSTGRES_DB: "databasename"
31     ports:
32         - "5432:5432" # (4) Port exposed to all networks.
33     volumes:
34         - ./postgres-data:/var/lib/postgresql/data # (7) Potential volume
security issue.
35     networks:
36         - app-network
37     healthcheck:
38         test: "pg_isready --username=somesafeuser --dbname=databasename && psql --
username=somesafeuser --list"

```

```
39     timeout: 3s
40     retries: 5
41     restart: unless-stopped # (6) May restart compromised containers
indefinitely.

42 networks:
43   app-network:
44     driver: bridge
```

Detailed Security Issues by Line Number

1. Line 4 – Using `latest` tag:

- Using `latest` can introduce breaking changes or vulnerabilities from unintended updates.

Mitigation: Use a specific version tag for predictable behavior (e.g., `deepset/haystack-annotation:1.0.0`).

2. Lines 9, 10, 28, 29 – Sensitive data in plaintext:

- Database credentials are hardcoded. If exposed, these can be exploited.

Mitigation: Use Docker **secrets** or an environment variable manager.

3. Lines 12-15 – Sensitive values in comments:

- Even commented-out credentials like `JWT_SECRET` and `DEFAULT_ADMIN_PASSWORD` can leak if the YAML is shared.

Mitigation: Replace with placeholders or remove them entirely (e.g., `<insert-password>`).

4. Lines 17, 32 – Ports exposed to all networks:

- Exposing ports like 5432 (database) and 7001 (backend) increases the attack surface.

Mitigation: Bind to localhost or internal IPs:

yaml

```
ports:
  - "127.0.0.1:7001:7001"
  - "127.0.0.1:5432:5432"
```

5. Line 21 – `depends_on` limitation:

- `depends_on` ensures only service startup order, not readiness. Backend may try to connect before the database is ready.

Mitigation: Use health checks or handle retries in your backend.

6. Lines 24, 41 – Unrestricted `restart` policy:

- `unless-stopped` could restart broken containers indefinitely, masking underlying problems.

Mitigation: Monitor service health and use alerting mechanisms.

7. Line 34 – Potential volume security risk:

- The host directory `./postgres-data` is mounted without ensuring proper permissions. This could allow unauthorized access or modification.

Mitigation: Ensure the host volume is securely configured, with restricted permissions, and consider encrypted volumes if necessary.

By addressing the vulnerabilities highlighted above, you can better secure your application and infrastructure.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Below is a detailed analysis of the potential vulnerabilities present in your YAML configuration, mapped to relevant **OWASP Top 10 Web Application Security Risks (2021)** categories.

1. Sensitive Data Exposure (OWASP A02:2021 - Cryptographic Failures)

Affected lines:

```
yaml
```

```
09      DB_USERNAME: "somesafeuser"
10      DB_PASSWORD: "somesafepassword"
```

```
28     POSTGRES_USER: "somesafeuser"
29     POSTGRES_PASSWORD: "somesafepassword"
```

- **Risk:** Sensitive credentials are stored in plaintext within the configuration. If this YAML file is leaked (e.g., through version control), attackers can easily gain access to the database or application.
- **Mitigation:**
 - Use **Docker secrets** or external secrets management services like **Vault**.
 - Never store passwords or secrets directly in version-controlled files.

2. Security Misconfiguration (OWASP A05:2021)

Affected lines:

```
yaml

04     image: deepset/haystack-annotation:latest
21     - db
32     - "5432:5432"
17     - "7001:7001"
```

- **Risk:**
 1. **Line 4:** Using the `latest` tag makes the system vulnerable to unforeseen changes, introducing security risks due to unintended updates or breaking changes.
 2. **Lines 21, 32, 17: Ports are exposed to all networks** (by default `0.0.0.0`), allowing external access to sensitive services, such as the database.
- **Mitigation:**
 - Pin image versions explicitly (e.g., `postgres:12.3`).
 - Use network restrictions to bind ports to `127.0.0.1` or internal IPs only.
 - Implement firewalls or **Docker network policies** to limit service access.

3. Broken Authentication (OWASP A01:2021)

Affected lines:

yaml

```
12      # DEFAULT_ADMIN_EMAIL: "example@example.com"
13      # DEFAULT_ADMIN_PASSWORD: "DEMO_PASSWORD"
15      # JWT_SECRET: "somesafesecret"
```

- **Risk:**
 - Even though these credentials are commented out, **leaked YAML files** containing admin credentials or JWT secrets could allow attackers to impersonate users, perform privilege escalation, or craft valid tokens.
 - JWTs with weak or exposed secrets increase the risk of **session hijacking** or **authentication bypass**.
- **Mitigation:**
 - Remove commented-out credentials from the YAML.
 - Use environment variables or **Docker secrets** to manage sensitive tokens securely.

4. Insecure Design / Lack of Security Controls (OWASP A04:2021)

Affected lines:

yaml

```
20      depends_on:
21        - db
41      restart: unless-stopped
```

- **Risk:**
 - Using `depends_on` (Line 20) ensures only **service startup order** but does not guarantee the database will be ready. This can lead to race conditions where the backend starts before the database is available, causing potential issues in the application.

- **Restart policy** on Line 41 could restart compromised or malfunctioning containers indefinitely, masking underlying security issues.
 - **Mitigation:**
 - Use **health checks** and **retry logic** in the backend to handle service dependencies correctly.
 - Monitor container restarts and trigger alerts on abnormal behavior to detect compromised services.
-

5. Insecure Data Storage (OWASP A03:2021 - Injection / A09:2021 - Security Logging and Monitoring Failures)

Affected lines:

yaml

```
34      - ./postgres-data:/var/lib/postgresql/data
```

- **Risk:** Mounting a host directory as a volume can expose sensitive data (such as database files) if the directory is not properly secured. An attacker with access to the host system could read or modify the database contents.
 - **Mitigation:**
 - Use **encrypted Docker volumes** for storing sensitive data.
 - Ensure that the host directory has appropriate **file permissions** to prevent unauthorized access.
-

6. Improper Restriction of Network Traffic (OWASP A05:2021 - Security Misconfiguration)

Affected lines:

yaml

```
32      - "5432:5432" # Exposes database port to all networks.
17      - "7001:7001" # Exposes application port to all networks.
```

- **Risk:** Ports are exposed to all networks, increasing the **attack surface**. An attacker could try to brute-force the database or exploit unpatched vulnerabilities.
- **Mitigation:**
 - Restrict port binding to `127.0.0.1` or internal IP addresses:

```
yaml

ports:
  - "127.0.0.1:5432:5432"
  - "127.0.0.1:7001:7001"
```

- Use **network firewalls** and **access control lists (ACLs)** to limit traffic.

7. Lack of Resource Management (OWASP A06:2021 - Vulnerable and Outdated Components)

Affected lines:

```
yaml

26      image: "postgres:12"
```

- **Risk:** Using outdated components or images (e.g., older Postgres versions) may introduce vulnerabilities due to **unpatched security flaws**.
- **Mitigation:**
 - Regularly **update container images** and dependencies.
 - Use vulnerability scanning tools to detect outdated components.

8. Insufficient Monitoring & Logging (OWASP A09:2021)

Affected lines:

yaml

```
38      test: "pg_isready --username=somesafeuser --dbname=databasename && psql --
username=somesafeuser --list"
41      restart: unless-stopped
```

- **Risk:**
 - The current setup lacks proper **monitoring** and **logging** for service health. If the database or backend fails, the application might restart without any visibility into the underlying issues.
- **Mitigation:**
 - Use **logging solutions** (e.g., ELK stack, Prometheus) to monitor container logs and application health.
 - Add **alerting mechanisms** to notify administrators on failures or unusual container behavior.

Summary of Vulnerabilities Mapped to OWASP Top 10 Categories

OWASP Top 10 Risk	Description	Relevant Lines
A01:2021 - Broken Authentication	Hardcoded admin passwords and JWT secrets	12-15
A02:2021 - Cryptographic Failures	Credentials stored in plaintext	9-10, 28-29
A03:2021 - Injection	Sensitive data exposed through improper volume management	34
A04:2021 - Insecure Design	Missing retry logic and service health checks	20-21, 41
A05:2021 - Security Misconfiguration	Exposed ports, unencrypted volumes, latest image tags	4, 17, 32, 34

OWASP Top 10 Risk	Description	Relevant Lines
A06:2021 - Vulnerable and Outdated Components	Use of potentially outdated Docker images	26
A09:2021 - Security Logging and Monitoring Failures	Lack of monitoring on container restarts	38, 41

Final Recommendations

By addressing the identified vulnerabilities and following the suggested mitigations, your application will be better protected against unauthorized access, data leaks, and operational disruptions. Be sure to:

1. Use **Docker secrets** for sensitive data.
2. Implement **network restrictions** to reduce attack surface.
3. Enable **monitoring and alerting** to detect abnormal behavior.
4. Avoid using the `latest` image tag to prevent unexpected updates.